

ENIGMA ENCRYPTION

Project Report

Custom password hashing & file encryption — C++ core · PyPI library · Streamlit demo

Repository	github.com/dsainvg/enigma
Live Demo	dev.dsainvg.me/enigma
Project Hub	projects.dsainvg.me/enigma
PyPI	pypi.org/project/enigma-encryption
Tech Stack	C++17 · pybind11 · Python · CMake · GitHub Actions
Date	June 2026

Project Overview

Enigma is a personal end-to-end engineering project: a custom password-hashing and file-encryption system, implemented in C++, exposed as a pip-installable Python library, and shipped with automated CI/CD across Linux, macOS, and Windows. A live Streamlit demo lets anyone try file encryption in the browser without installing anything.

The project's defining constraint was that all cryptographic logic lives exactly once — in a shared C++ core — and both the Python library and the CLI binary are compiled from that same source. This prevents the algorithm drift that caused an earlier iteration to be abandoned and became the central architectural decision of the final design.

```
pip install enigma-encryption

import enigma
enc = enigma.encrypt_bytes(data, 'password', cost=10)
dec = enigma.decrypt_bytes(enc, 'password')
enigma.encrypt_file('doc.pdf', 'doc.enc', 'password', cost=10)
```

What Was Built & Shipped

Deliverable	Where	Status
Python library	pypi.org/project/enigma-encryption	✓ Published
CLI binary (Linux/macOS/Win)	GitHub Releases	✓ Published
Streamlit demo	dev.dsainvg.me/enigma	✓ Live
Project page	projects.dsainvg.me/enigma	✓ Live
Documentation site	dev.dsainvg.me/enigma (user + dev portal)	✓ Deployed
CI pipeline	GitHub Actions — 3 OSes on every push	✓ Active
Test suite	C++ (CTest) + Python pytest — 50+ tests	✓ Passing
Benchmark suite	Full sweep: file sizes, threads, buffers	✓ Complete

Development Journey

Enigma did not start as a C++ project. It went through four distinct phases across roughly eleven months, each one a deliberate engineering decision informed by what the previous phase proved or failed to prove.

Phase 1 — Python Streamlit Prototype (Aug 2025)

The first version was a pure-Python Streamlit web app. Its purpose was to validate the algorithm and the user experience quickly, not to be fast. It established the full encrypt / decrypt pipeline: password hashing with configurable cost factor, 1 KB block cipher, chunked file processing, and ZIP-based output. The custom hash algorithm and cipher were both designed and proven correct here.

What it proved: the algorithm works and the UX is intuitive.

What it exposed: Python hashing at cost=11 dominated total encryption time; not viable at scale.

Phase 2 — Desktop GUI Application (Oct 2025 — Abandoned)

An attempt to ship a polished glassmorphism desktop app: C++ backend via pybind11, Tkinter frontend with Poppins/Roboto fonts, multi-threaded file processing with memory-mapped I/O. The C++ backend was completed and benchmarked. The project was abandoned because:

- Decryption was unfinished — a placeholder function was never replaced.
- The build was hardcoded to Python 3.14 and Visual Studio 2022, making it non-portable.
- No packaging: the compiled extension had to be manually placed alongside the GUI script.
- Tkinter's limitations made the glassmorphism design unachievable without rendering hacks.

This phase is preserved as a historical record. Its self-generated codebase analysis documents exactly what failed and why — a deliberate engineering habit of honest post-mortems.

Phase 3 — C++ CLI (Dec 2025)

A clean C++ reimplement of the full algorithm with a working CLI. Three meaningful engineering improvements over the prototype:

- **Adaptive chunk sizing:** files under 5 MB are processed in one pass; larger files use progressively larger chunks (up to 8 MB), reducing per-chunk overhead at scale.
- **Multi-threaded encryption:** `std::thread` pool divides chunk work across available cores.
- **Fast-reject on wrong password:** a hash-of-hash stored in the file header is checked before touching any ciphertext. Wrong passwords rejected in milliseconds.

190 automated tests — hash correctness, cross-check against Python, round-trip encryption at multiple file sizes and cost factors.

Phase 4 — Enigma Monorepo (Jun 2026 — Current)

The final repository unified everything. The C++ core is the single source of truth. The Python library and the CLI binary are both compiled outputs of the same CMake project. A git tag push triggers both PyPI publication and GitHub Release binaries simultaneously.

Architecture

Monorepo Structure

The architecture is deliberately flat. There is no shared state between the Python surface and the CLI; they are independently compiled consumers of the same C++ headers and source files.

C++ Core (core/)

- hash.h / hash.cpp — iterative password hashing with memoisation table
- cipher.h / cipher.cpp — block rotate-and-XOR symmetric cipher
- primitives.cpp — bitwise rotation across arbitrary-length buffers
- salt.cpp — CSPRNG salt generation
- file_io.cpp — streaming multi-threaded file encrypt/decrypt

Consumers

- cli/ — enigma binary, published to GitHub Releases
- python/ — pybind11 bindings, packaged as PyPI wheel via scikit-build-core
- tests/cpp/ — CTest suite
- tests/python/ — pytest suite (50+ tests)
- benchmarks/ — sweep harness (file size × threads × buffer)

Hashing Algorithm

Custom iterative password-hashing with a configurable cost factor. The algorithm runs 2^{cost} rounds; each round produces intermediate values stored in a growing memoisation table. Subsequent rounds index into that table at password-derived offsets, forcing each iteration to depend on all prior outputs and requiring RAM proportional to iteration count — a property that reduces the advantage of GPU-parallel brute-force attacks.

Output format embeds the salt: `$<salt>$/<hash>`. The salt is always recoverable from the hash string, so password verification requires only re-hashing — no separate salt storage.

Cipher

Files and byte buffers are divided into 1 KB blocks. Each block is encrypted with 10–15 rounds (the exact count varies by block index, so no two blocks are processed identically). Each round: rotate the entire block bitwise left by a key-derived shift, then XOR every byte with the corresponding key byte. Decryption is the exact reverse.

File Format — ENIGMA01

Custom binary format with no external library dependency. The file header carries the salt, cost factor, and a password verification tag (hash of the derived key). On decryption, the verification tag is checked first — wrong passwords rejected before any ciphertext is touched.

Header Field	Purpose
ENIGMA01 magic (8 B)	File type identification
Salt (16 B) + Cost (2 B)	KDF parameters — self-contained, no external storage needed
Verification tag (variable)	Hash-of-hash; decryption aborts here on wrong password
Ciphertext blocks	1 KB sequential encrypted blocks

Performance

All benchmarks were measured on a 4-core CPU running Python 3.13.9 on Windows. Each cell is a 5-run average. The results below are the actual numbers from the project's benchmark suite — not estimates.

Password Hashing — Full Matrix

The cost factor is the primary control. Every +2 to cost multiplies runtime by ~4 (2^{cost} scaling). Password length has negligible effect — a 512-character password at cost=10 hashes in 57 ms; an 8-character password takes 57 ms too.

Password Length	Cost	Avg Time (ms)	Min (ms)	Max (ms)	Throughput (h/s)
8 chars	8	14.2	14.0	14.5	70.4
16 chars	8	13.5	10.7	18.1	73.9
64 chars	8	12.2	10.9	13.0	82.3
256 chars	8	17.3	12.4	30.4	57.8
512 chars	8	21.2	15.0	36.7	47.1
8 chars	10	56.6	55.2	58.6	17.7
32 chars	10	49.1	46.9	54.2	20.4
64 chars	10	49.1	48.2	50.1	20.4
128 chars	10	50.6	49.0	54.7	19.8
256 chars	10	51.4	48.5	57.2	19.4
512 chars	10	57.4	51.6	77.5	17.4
8 chars	12	224.1	222.5	225.4	4.5
64 chars	12	196.4	193.8	200.6	5.1
256 chars	12	211.8	199.5	236.4	4.7

Password Length	Cost	Avg Time (ms)	Min (ms)	Max (ms)	Throughput (h/s)
512 chars	12	206.2	199.6	224.9	4.9
8 chars	14	911.8	907.6	917.4	1.1
64 chars	14	878.5	860.6	892.7	1.1
256 chars	14	873.7	858.0	905.8	1.1
512 chars	14	893.4	867.6	971.6	1.1

Cost=8 → ~14 ms | Cost=10 → ~57 ms | Cost=12 → ~210 ms | Cost=14 → ~900 ms
 Each +2 to cost = ×4 slower. Password length makes no measurable difference.

File Encryption — Threading & Throughput

Cipher work is fully parallelisable across 1 KB blocks. The tables below show how throughput scales with thread count on real files. Cost=8 throughout; KDF adds ~14 ms overhead on top of these numbers regardless of file size.

1 MB file (cost = 8, buffer = 1 MiB)

Threads	Encrypt MB/s	Decrypt MB/s	Peak RAM
1	8.9	8.7	10.7 MB
2	15.5	12.8	1.1 MB
4	20.6	14.6	1.1 MB

64 MB file (cost = 8)

Threads	Buffer	Encrypt MB/s	Decrypt MB/s	Peak RAM
1	1 MiB	12.9	12.7	1.0 MB
2	4 MiB	26.6	26.9	4.2 MB
4	4 MiB	53.8	48.8	4.1 MB
4	16 MiB	49.6	49.2	16.1 MB

256 MB file (cost = 8)

Threads	Buffer	Encrypt MB/s	Decrypt MB/s	Peak RAM
1	1 MiB	12.9	12.7	1.0 MB
2	4 MiB	25.8	25.2	4.1 MB
4	4 MiB	51.3	49.9	4.1 MB
4	32 MiB	50.0	50.0	32.1 MB

500 MB file (cost = 8)

Threads	Buffer	Encrypt MB/s	Decrypt MB/s	Peak RAM
1	1 MiB	13.1	12.8	1.0 MB
2	4 MiB	25.8	25.7	13.2 MB
4	4 MiB	50.4	50.3	4.1 MB
4	32 MiB	48.5	48.6	32.1 MB

Key Observations

- **Linear thread scaling:** throughput doubles from 1→2 threads and doubles again from 2→4 threads on all file sizes above 1 MB. The cipher is CPU-bound and independently parallelisable.
- **Constant memory:** peak RAM tracks buffer size, not file size. A 500 MB file encrypted with a 4 MiB buffer uses 4.1 MB peak RAM — memory footprint is fully controlled by the caller.
- **KDF dominates tiny files:** for 1 KB files, total time is 30–120 ms regardless of thread count or buffer — all of it is the key derivation step. Thread count has zero effect for files under ~1 MB.
- **Buffer tuning:** larger buffers (16–32 MiB) improve single-thread throughput by ~30% due to reduced syscall overhead, but the memory tradeoff is linear. For multi-threaded workloads, 4 MiB is optimal.

Choosing a Cost Factor

Cost	Time (4-core)	Iterations	Recommended For
8	~14 ms	256	Development / testing
10	~57 ms	1024	Default — general use, good balance
12	~210 ms	4096	Sensitive files, interactive tolerable
14	~900 ms	16384	Long-term archives, infrequent access

Key Engineering Decisions**Single source of truth**

Algorithm drift between Python and C++ caused the Phase 2 application to be abandoned. The monorepo design ensures the Python library and CLI binary are compiled outputs of identical source — not independent reimplementations.

Zero external crypto dependencies

The final library has no runtime dependency on OpenSSL, libzip, or any crypto library. This eliminates supply-chain attack vectors and makes the wheel self-contained on all three platforms.

Trusted Publishing (no stored secrets)

PyPI deployment uses OIDC Trusted Publishing. GitHub's identity token authenticates directly to PyPI — no API tokens in repository secrets. A single git tag push publishes both the PyPI wheel and the GitHub Release binaries.

Honest failure documentation

The abandoned GUI repository is preserved with a full post-mortem. The decision to restart rather than patch is documented alongside the technical reasons — a deliberate engineering habit, not an accident.

Security Posture

Enigma is transparent about what it is and what it is not. The security documentation is explicit on both sides.

What it provides

- **Work factor:** $2^{\wedge}$ cost iterations make brute-force dictionary attacks computationally expensive and user-configurable.
- **Memory-hard element:** each hashing iteration reads from a growing memo table at password-derived offsets, reducing GPU/FPGA parallelism advantage.
- **Per-file random salt:** CSPRNG-generated, embedded in the output file, preventing precomputed attacks.
- **Fast wrong-password rejection:** verification tag in file header checked before any ciphertext is decrypted.

What it does not provide

No authenticated encryption (AEAD) — file tampering produces corrupted output, not a verified error.

No peer-reviewed cryptanalysis — the algorithm is custom and has not been independently audited.

Not recommended for: regulated data, financial systems, health records, or production authentication.

For those use cases: AES-256-GCM + Argon2id via PyNaCl or Python's cryptography package.

These limitations are documented prominently in the project README and the dedicated security page — not buried in footnotes.

Summary

Enigma is a complete engineering arc from idea to shipped product. The project demonstrates the ability to:

- **Design and iterate an algorithm** across multiple languages and delivery mechanisms while keeping correctness as the constant.
- **Make architectural pivots** when an approach proves unworkable — twice — and document why rather than sunk-cost continuing.
- **Ship to real distribution channels:** PyPI with OIDC Trusted Publishing, GitHub Releases with multi-platform binaries, a live hosted demo.
- **Write a full CI/CD pipeline** that builds, tests, and publishes across Linux, macOS, and Windows with zero manual steps after a tag push.
- **Benchmark systematically:** not just 'it's fast' — a sweep across file size, thread count, and I/O buffer size with measured throughput and peak memory.
- **Be honest about limitations:** security caveats documented up front, not discovered by users.